Draft: A Survey of Type Theories: Martin Löf, Homotopy and Cubical

A Short Review

Naveen Maurya

April 14, 2026

1 Introduction

Type theory is a family of foundational systems, many based on a dependently typed λ-calculus. The Curry-Howard correspondence links proofs and programs and allows theorem provers like Coq, Agda, Scala, Lean etc. powered by type theories, to assist in machine-checking proofs in a constructive and computational framework. [8] In this report, we shall describe (intensional) instuistionistic Martin-Löf Type Theory, Homotopy Type Theory and Cubical Type Theory in brief detail.

2 Basic Description

We shall describe a standard version of type theory called (intensional) Martin-Löf Type Theory (MLTT [11]). Set theoretic foundations usually consist of a number of *axioms*, written in the language of first-order logic, which is separately assumed beforehand. The logic's deductive system provides rules for making inferences, while the *axioms* define and describe sets and their semantics. This forms a basis to build mathematics, using sets to describe mathematical objects, while manipulating statements via first-order logic. MLTT conserves the dependence on axioms and constructs more machinery within the deductive system itself.The deductive system consists of the primitive notion of 'types', the building blocks for mathematical objects as well as the logic, while specifying the rule that govern them.

MLTT consists of *rules* for deriving *judgements*, which are either of the form $a : A$ meaning 'a is a term/instance of type A', or $a \equiv b : A$ meaning 'a is judgementally equal to b', while both being instances of A. Judgemental equality is a strict notion of equality which classifies objects as definitionally the same within the intrinsic language of the formal system. A separate notion 'propositional equality', denoted with $=$, is a weaker notion of equality, corresponding to the equality of the mathematical objects to be described. This is only a feature of *intensional* MLTT.

The language of MLTT is an extension of dependently-typed λ-calculus, with variables $x_1, x_2, \ldots$, primitive constants $c, c', c'', \ldots$, and defined constants f, f', f''. The terms $t, t', t'', \ldots$ are given by the grammar

$$t \to x \mid \lambda x.t \mid t(t') \mid c \mid f$$

Further, MLTT provides rules to form types from a pre-existing context, and rules to manipulate defined or pre-existing types to encode their semantics.

The logic of MLTT comes encoding propositions as types, wherein having an term of a 'proposition as a type' is interpreted as a witness or evidence of the proposition. [2] MLTT being an intuitionistic formal system, has a constructive logic, which requires any existence proof to be certified by a witness. [11]

2.1 Universes

Type Theory 'postulates' a chain of cumulative universes à la Russell, $\mathcal{U}_0, \mathcal{U}_1, \mathcal{U}_2, \ldots$ as primitive constants with

- $\mathcal{U}_i : \mathcal{U}_{i+1}$ for each i
- If $A : \mathcal{U}_i$, then $A : \mathcal{U}_{i+1}$

Any type in the theory is an instance of some universe in this chain. Each universe is also a type, and an instance of universes above it in this heirarchy. But unless necessary, we omit the index from the universe and write $A : \mathcal{U}$.

Remark. *This chain, while indexed by natural numbers, is not related to the Natural Numbers that will be defined in the theory. The indices come the metatheory. We shall see the arithmetic object of Natural Numbers ($\mathbb{N}$) is richer and needs more information to be properly defined.*

3 Inductive Types

Type Theory provides deductive rules to give schemas to construct, manipulate, and eliminate various constructions. A number of these constructions are *synthetically* given rules in the deductive system, known as *type formers*. Type Theories also allow the user to define their own constructions known as inductive types, with a similar schema.

The general scheme for these type formers is as follows. An type former is defined by:

- A list of ***formation rules***, describing how the inductive type can be introduced into context.
- A list of ***constructors/indroduction rules***, to introduce new terms of these types.
- ***Elimination rules*** to give a recipe how to use/consume terms of the type. Usually, this involves a *recursion principle* to define functions on the type, and a stronger *induction principle* to define dependent functions on the type. Both work by reducing the task to define the function on a few special cases, given by the introduction rules.
- A ***computation rule***, dictating how the elimination acts on the constructors.
- (Optional) ***uniqueness principles***, dictating how each term of the type is uniquely determined by the result of elimination rules applied to it.

Here is a succinct, informal description of these type formers in Type Theory. They may also be interpretated as logical objects under 'propositions as types' interpretation. A detailed, formal description is avaiable in Appendix I of [13].

3.1 Some Type Formers/Inductives

3.1.1 Function Types

Given $A, B : \mathcal{U}$, we can form $A \to B : \mathcal{U}$, the type of functions from A to B, syntactically given by a simple λ-calculus. Introducing a function can be done by specifying a formula as $(\lambda x.\Phi)$ where Φ is a formula in the variable x. Elimination is performed by evaluation, computed as $(\lambda x.\Phi)(a) \equiv \Phi[a/x]$, replacing all occurences of x with a. We get $f(a) : B$ for any $f : A \to B$ and $a : A$.

Iterated function types such as $A \to (B \to C))$ are effectively multivarible function types (due to Product-Exponential adjunction), so the parentheses are omitted and associated from right by default, like $A \to B \to C$. This is called *currying*. Currying is also used in lambda notation, writing $(\lambda x.\lambda y.\Phi)$ instead of $(\lambda x.(\lambda y.\Phi))$.

Logically, $A \to B$ correponds to $A \implies B$, albeit constructively. That is, to show $A \implies B$, one must construct an explicit function taking terms of type A to terms of type B.

3.1.2 Dependent Function Types (Π-types)

Given $A : \mathcal{U}, B : A \to \mathcal{U}$, we can form the type of dependent functions $\prod_{(x:A)} B(x) : \mathcal{U}$. Its terms are functions

with codomain dependent on the domain. They follow similar rules as regular function types, but $f(a) : B(a)$ for any $a : A$ and $f : \prod_{(a:A)} B(a)$.

Logically, $\prod_{(x:A)} B(x)$ corresponds to the predicate $\forall x \in A, (B(x))$ and can be read as such, except it needs to be constructive.

3.1.3 Product Types

Given $A, B : \mathcal{U}$, we can form the product type $A \times B : \mathcal{U}$ consists of pairs (a, b), given by $a : A, b : B$. Let $E : A \times B \to \mathcal{U}$, then to define $f : \prod_{(z:A\times B)} E(z)$, it suffices to define f for the special case where $x \equiv (a, b)$ for some $a : A, b : B$. This is the *induction principle*, denoted $\mathsf{ind}_{A\times B}$. The computation rule enforces the defined function matches in the special case.

Logically, $A \times B$ corresponds to $A \wedge B$.

3.1.4 Coproduct Types

Given $A, B : \mathcal{U}$, we can form the coproduct type $A + B : \mathcal{U}$ given by two constructors $\mathsf{inl} : A \to A + B$, and $\mathsf{inr} : B \to A + B$. For its induction principle, to define a function $f : \prod_{(z:A+B)} E(z)$, it suffices to define f for the special case where $z \equiv \mathsf{inl}(x)$ and the case where $z \equiv \mathsf{inr}(y)$.

Logically, $A + B$ corresponds to $A \vee B$.

3.1.5 Dependent Pair Types (Σ-types)

This is the dependent version of a product type. Given $A : \mathcal{U}, B : A \to \mathcal{U}$, we can form $\sum_{(a:A)} B(a) : \mathcal{U}$. Say $a : A, b : B(a)$, one can form $(a, b) : \sum_{(a:A)} B(a)$. Let $E : \sum_{(a:A)} B(a) \to \mathcal{U}$. To define $f : \prod_{(c:\sum_{(a:A)} B(a))} E(c)$, it suffices to define f in the special case where $c \equiv (a, b)$ for some $a : A, b : B(a)$.

Logically, $\sum_{(x:A)} B(x)$ corresponds to the predicate $\exists x \in A, (B(x))$, and may be read as such barring constructivity.

3.1.6 0, 1 and 2

The empty type $\mathbf{0}$ is the type with no constructors, and the induction principle, that to define $f : \prod_{(x:\mathbf{0})} E(x)$, it suffices to *not define anything*. By default, we have a function, $\prod_{(x:\mathbf{0})} E(x)$. The unit type $\mathbf{1}$ is the type with a single constructor, $\star : \mathbf{1}$. The induction principle says that to define $f : \prod_{(x:\mathbf{1})} E(x)$, it suffices to define f for $x \equiv \star$. The boolean type $\mathbf{2}$ is the type with two constructors, $0_2 : \mathbf{2}$ and $1_2 : \mathbf{2}$. To define $f : \prod_{(x:\mathbf{2})} E(x)$, it suffices to define for $x \equiv 0_2$ and $x \equiv 1_2$. Alternatively, one can define $\mathbf{2} :\equiv \mathbf{1} + \mathbf{1}$.

Logically, $\mathbf{0}$ is a contradiction ($\bot$) and $\mathbf{1}$ is True ($\top$). However, $\mathbf{2}$ does not have any equivalent in first-order logic. In can be shown that MLTT has a richer logic than first-order under this correspondence.

3.1.7 Natural Numbers Type ($\mathbb{N}$)

The natural numbers type is the prototypical example of an *Inductive Type*. It is generated by the constructors

- $\mathsf{zero} : \mathbb{N}$
- $\mathsf{succ} : \mathbb{N} \to \mathbb{N}$

Let $E : \mathbb{N} \to \mathcal{U}$. Then to define $E(n)$ for all $n : \mathbb{N}$, it suffices to define it for the following special case $n \equiv \mathsf{zero}$, and for all $m : \mathbb{N}$, given $f(m)$ define it for $n \equiv \mathsf{succ}(m)$. (This is almost the usual principle of mathematical induction!)

3.1.8 Identity Types

Given $A : \mathcal{U}$, we can form the identity type family $\mathsf{Id}_A : A \to A \to \mathcal{U}$. Given $x, y : A$, we have $\mathsf{Id}_A(x, y)$, also denoted as $x =_A y$, which is the type of all *propositional equalities* between x and y. The introduction rule is by using $\mathsf{refl} : \prod_{(a:A)} (a =_A a)$. Identities being types themselves, we can form iterated identity types such as $\mathsf{Id}_{\mathsf{Id}_A(x,y)}(p, q)$ of eqaulities between $p, q : x =_A y$. Intensional MLTT has *proof relevance* meaning one allows multiple proofs of identity types to not be equal, making it sensical to question about iterated identity types. This way, proofs themselves become mathematical objects, susceptible to analysis and proof.

The induction principle for Identity Types is called path induction, which says given $C : \prod_{(x,y:A)} (x =_A y) \to \mathcal{U}$, a function $c : \prod_{(x:A)} C(x, x, \mathsf{refl}_x)$, there is a function $f : \prod_{(x,y:A)} \prod_{(p:x=_A y)} C(x, y, p)$ such that $f(x, x, \mathsf{refl}_x) :\equiv c(x)$. That is, to define a function on paths between points in A, it suffices to exhibit a function on all reflexive loops refl_x over all points $x : A$.

3.2 Custom Inductive Types

Type Theories provide a schema to build newer, user-defined inductive types as well. Conceptually, an inductive type is a type *freely generated* by a number of constructors, which may involve functions that output the given type by (possibly) taking as input terms of the same type. This intuition needs to be formalized carefully so as to ensure that only meaningful inductive types may be constructed.

Remark. *Inductive types are objects one can perform induction on! Similarly to $\mathbb{N}$, the induction principle for a given inductive type is the statement that to prove a statement for all its instances, it suffices to prove it in a few special cases, built on the constructors of the type.*

3.2.1 W-Types

There is a complete formalism of inductive types up to certain demands for the constructors, first appearing in [12].

Definition 1 (W-Types)**.** *Given $A : \mathcal{U}, B : A \to \mathcal{U}$, the W-Type $\mathsf{W}_{(a:A)}B(a)$ is defined by the constructor*

$$\mathsf{sup} : \left(\prod_{(a:A)} B(a) \to \mathsf{W}_{(a:A)}B(a) \right) \to \mathsf{W}_{(x:A)}B(x)$$

and the induction principle, to prove $E : \mathsf{W}_{(a:A)}B(a) \to \mathcal{U}$ for all elements of $\mathsf{W}_{(a:A)}B(a)$, it suffices to prove it for $\mathsf{sup}(a, f)$ for each $a : A$, and $f : (B(a) \to \mathsf{W}_{(a:A)}B(a))$, assuming it holds for all $f(b)$ with $b : B(a)$.

A W-Type $\mathsf{W}_{(a:A)}B(a)$ may be interpreted as the type of well-founded trees with A-many ways to branch out from each node, corresponding to a constructor. Each $a : A$ corresponds to a constructor, each of which take $B(a)$-many instances of $\mathsf{W}_{(a:A)}B(a)$ as branches. The type $\mathsf{W}_{(a:A)}B(a)$ is the type of objects *freely generated* by these constructors. If an inductive type may be expressed equivalently

as generated by sup as described, it can be formalized as a W-Type.

- The boolean type $\mathbf{2}$ is equivalent to the W-Type $\mathsf{W}_{(a:A)}B(a)$ where $A \equiv \mathbf{2}$ with $B(0_2) \equiv B(1_2) \equiv \mathbf{0}$, that is generated by two constructors, $0_2 : \mathbf{2}$ and $1_2 : \mathbf{2}$.
- The natural numbers type $\mathbb{N}$ is equivalent to $\mathsf{W}_{(a:A)}B(a)$ where $A \equiv \mathbf{2}$, with $B(0_2) \equiv \mathbf{0}$ and $B(1_2) \equiv \mathbf{1}$, i.e., generated by two constructors, corresponding to $\mathsf{zero} : \mathbb{N}$, and $\mathsf{succ} : \mathbb{N} \to \mathbb{N}$.
- The type $\mathsf{W}_{(x:\mathbf{1}+A)}B(x)$ with $B(\mathsf{inl}(\star)) \equiv \mathbf{0}$ and $B(\mathsf{inr}(a)) \equiv \mathbf{1}$, corresponds to the type $\mathsf{List}(A)$ of finite length strings over the alphabet A, i.e. generated by $\mathsf{nil} : \mathsf{List}(A)$ and $\mathsf{cons} : A \to \mathsf{List}(A) \to \mathsf{List}(A)$.

4 Homotopy Type Theory

Homotopy Type Theory (HoTT) extends MLTT by adopting a homotopical interpretation of types. It is enriched by principles such as the *univalence axiom* and *higher inductive types*, enabling the direct construction of spaces with specified higher path structure and supporting reasoning up to homotopy.

4.1 A Homotopical Intepretation

In HoTT, types can be interpretated either topologically as spaces, or as (weak) ∞-groupoids from higher-dimensional category theory. A topological space weakly homotopy equivalent to a CW-complex can be classified upto homotopy equivalence by considering its *fundamental (weak) ∞-groupoid*. [10, 14] This follows from the fundamental ∞-groupoid construction being adjoint to the geometric realization functor. (This is the *homotopy hypothesis/theorem*, introduced by Grothendieck in *Pursuing Stacks* [6]).

4.2 Identity Types

The precise manner in which types correspond to spaces or ∞-groupoids is demonstrated by establishing an associated ∞-groupoid.

In the homotopy interpretation, terms of a type are thought of a points, and the identity type $\mathsf{Id}_A(x, y)$ corresponds to the space of paths from $x : A$ to $y : A$ endowed with the compact-open topology. This is exactly the free path space PA. [16] refl_x is the constant path at x. The iterated path type, $\mathsf{Id}_{\mathsf{Id}_A(x,y)}(p, q)$ corresponds to the iterated free path space P^2A. Note that $h : \mathsf{Id}_{\mathsf{Id}_A(x,y)}(p, q)$ are homotopies from p to q. And similarly the n-th iterated path type corresponds to the n-th path space P^nA.

We can extract the (weak) ∞-groupoid structure from this observation with a few arguments.

Let $\mathcal{U}$ be fixed and $A : \mathcal{U}$. Define $\mathsf{Id}^0(A) :\equiv A$ and $\mathsf{Id}^{\mathsf{succ}(n)} :\equiv \sum_{(x,y:\mathsf{Id}^n(A))} (x = y)$

Theorem 1. *There is a functorial association of types to weak ∞-groupoids, where $A : \mathcal{U}$ maps to the infinity groupoid where n-th level objects are terms of $\mathsf{Id}^n(A)$.*

The idea of the proof is to construct inverse and concatenation operations at a fixed level, and showing they follow unit, inversion and associativity laws upto terms of a higher level (via path induction). Finally, showing functoriality (across independent and dependent functions) also follows from path induction. A full walkthrough for this is laid out in Chapter 2 of [13].

4.3 Axioms in HoTT

Some expected semantic behaviour does not directly follow from the machinery of HoTT laid out so far. To enforce such behaviour we introduce a few 'axioms' into our theory, which are merely inhabitants of some type without an inductive construction. But first, we make some definitions.

Definition 2 (Homotopy of maps)**.** *Let $f, g : \prod_{(x:A)} B(a)$, then we say f is homotopic to g, denoted*

$$(f \sim g) :\equiv \prod_{(x:A)} (f(x) =_{B(x)} g(x))$$

Definition 3 (Equivalence)**.** *Let $A, B : \mathcal{U}$. We say A and B are (homotopy) equivalent if there is an inhabitant of*

$$(A \simeq B) :\equiv \sum_{(f:A\to B)} \sum_{(g:B\to A)} (g \circ f \sim \mathsf{id}_A) \times (f \circ g \sim \mathsf{id}_B)$$

The notion of equality for functions is too restrictive to capture their expected behaviour. Classically, we expect two functions to be equal if they take the same values on all the inputs. To ensure semantic well-behavedness, we must enforce this as an axiom.

Axiom 1 (Function Extensionality)**.** *Let $A : \mathcal{U}, B : A \to \mathcal{U}$ and $f, g : \prod_{(x:A)} B(a)$, then we have $(f = g) \simeq (f \sim g)$.*

That is, equality of maps is characterized entirely by their values in the codomain. In particular, if two functions are homotopic, they are equal.

In [17], Voevodsky proposed the first variant of the univalence axiom.

Axiom 2 (Univalence)**.** *For $A, B : \mathcal{U}$, there is an equivalence: $(A =_{\mathcal{U}} B) \simeq (A \simeq B)$*

In particular, if $A \simeq B$, then $A =_{\mathcal{U}} B$. Roughly speaking, this means "isomorphic objects are equal". In a univalent universe $\mathcal{U}$, isomorphic objects/equivalent types can be treated as equal and one can transport (theorems, proofs and properties) across them.

4.4 Higher Inductive Types

We can now expand to types generated by constructors along with relations. We let the constructors output higher-dimensional paths in the type. This way, free generation is equivalent to generation on points quotiented by equivalence relations corresponding to these higher paths.

Such a *higher inductive type* primitive in HoTT is the circle type $\mathbb{S}^1$ generated by the constructors

- A point $\mathsf{base} : \mathbb{S}^1$
- A path $\mathsf{loop} : \mathsf{base} =_{\mathbb{S}^1} \mathsf{base}$

This corresponds to a chosen basepoint base, and a nontrivial path loop connected the basepoint, giving a circle.

Many familiar mathematical objects such as the integers, CW-complexes etc. can be built using this schema. Various approaches exist to formalize such a schema, but have only yeilded partial satisfaction. Some of the existing approaches are using Quotient Inductive Types [1], Homotopy Initial Algebras [15] , and computationally most satisfactory, Cubical Type Theory [5, 4]. A completely satisfactory formalism remains an open problem.

5 Cubical Type Theory

HoTT fails to achieve desirable computational behaviour. In particular, Univalence is an axiom and Canonicity is not guaranteed for HITs. HoTT affords presheaf model in simplicial sets. A computational model of type theory is then motivated using cubical sets in place for simplicial sets.

5.1 Cubical Sets

Let there be a fixed disrete and countable set of *names* $\mathbb{A}$ and define the set of *directions* $\mathbf{2} = \{0, 1\}$.

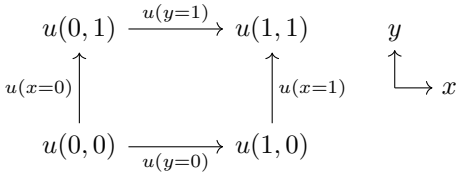
Definition 4 (The cubical category $\square$)**.** *[Coq] The cubical category $\square$ has as objects finite subsets $I, J, K... \subseteq \mathbb{A}$. A morphism $f : I \to J$ is a set function $f : I \to J \cup \mathbf{2}$, such that $fx = fy$ implies $x = y$ for $x, y \notin \mathbf{2}$. The composition of morphisms $f : I \to J$ and $g : J \to K$ is defined as $g \circ f(x) = g(f(x))$ when $fx \notin \mathbf{2}$ and fx otherwise.*

We will call morphisms *substitutions* as they encode sending a name to a direction. We will also denote fg for $g \circ f$ as well as xf for $f(x)$ from now on (called diagram order), and often omit braces in $I \cup \{x, y\}$ or $I - \{x, y\}$ for finite sets of names.

For $I \subseteq J$, we call the inclusion $f : I \to J$ a *degeneracy map*. We have the *face maps* $(x = 0), (x = 1) : I \to I - x$ sending x to 0 and 1 respectively. (Note how these constructions parallel the simplex category Δ)

Definition 5 (Cubical sets)**.** *A cubical set X is a covariant functor $X : \square \to \mathbf{Set}$, i.e., a presheaf over $\square^{op}$.*

That is, a cubical set X is a collection of sets $X(I)$ and for any substitution $f : I \to J$, set functions $X(I) \to X(J)$, $u \mapsto uf$ such that $u\mathbf{1} = u$ and $u(fg) = (uf)g$. This can be thought of as $X(\varnothing)$ is the set of points of a space, $X(x)$ are its lines, $X(x, y)$ are squares, and so on.



Cubical sets form a presheaf category, i.e. $\mathbf{cSet} := [\square, \mathbf{Set}]$ with morphisms given by natural transformations.

Example 5.1. *Let A be a set. Then $X : \square \to \mathbf{Set}$ with $X(\varnothing) = A$ and $X(I) = \varnothing$ if $I \neq \varnothing$ is called the discrete cubical set over A.*

Example 5.2. *Let R be a commutative ring. Then $R[\cdot]$ is a cubical set, with $R[I]$ being the polynomial ring over I, and for $f : I \to J$, $R[f](x) = f(x)$ where 0 and 1 in $\mathbf{2}$ are swapped for 0 and 1 from R.*

Example 5.3 (Interval)**.** *The interval $\mathbb{I}$ is the cubical set defined by $\mathbb{I}(I) = I \cup \mathbf{2}$ and for $f : I \to J$, $\mathbb{I}(f)$ is the extension of the set theoretic map of f such that $\mathbb{I}(f)(0) = 0$ and $\mathbb{I}(f)(1) = 1$.*

Note that for a fixed name x, $\mathbb{I} \cong \mathbf{y}\{x\}$, where $\mathbf{y}$ is the Yoneda embedding. More generally, $\square_I = \mathbf{y}I$ where I contains n elements, is the representable n-cube (since $\mathbf{y}I \cong \mathbf{y}J$ for $I \cong J$).

Definition 6 (Non-dependent Path Space)**.** *Given a cubical set X, we define its* path space $[\mathbb{I}]X$ *as a cubical set $([\mathbb{I}]X)(I) = X(I, x_I)$ where x_I is a fresh name $(x_I) \notin I$. Let $f : I \to J$ be a sustitution, then $X(f)$ is given by the induced map from f via inclusion, composed with substituting x_I for x_J, i.e., $\omega(X(f)) = \omega f(x_I = x_J)$.*

For a cubical set X, $[\mathbb{I}]X$ is also a cubical set. $[\mathbb{I}]X(\varnothing)$ are the points, $[\mathbb{I}]X(x)$ are the lines, $[\mathbb{I}]X(x, y)$ are the squares, and so on.

Definition 7 (Non-dependent Identity Type)**.** *Given a cubical set X, and $a, b \in X(\varnothing)$, the* identity type $\mathsf{Id}_X(a, b)$ *is the subobject of $[\mathbb{I}]X$ such that $\omega \in (\mathsf{Id}_X(a, b))(I)$ iff there is some name $x \in I$ such that $\omega(x = 0) = a$ and $\omega(x = 1) = b$.*

5.2 The Uniform Kan Condition

Definition 8 (Open boxes)**.** *Let X be a cubical set. Let I be a finite set of names, and $x, J \subseteq I$, with $x \notin J$. Fix a either 0 or 1. An* open box *denoted $\vec{u}$ is a family of faces u_{yb} in $X(I - y)$ for each (y, b) either (x, a) or $y \in J, b = 0, 1$. such that*

$$u_{yb}(z = a) = u_{za}(y = b)$$

for all $y, z \in J, y \neq z$ and $b, c = 0, 1$.

The idea is that an open box contains all faces, except for one face in a direction x.

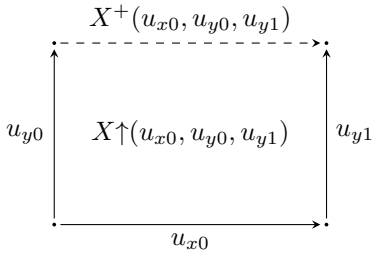
Definition 9 (Kan Cubical Set)**.** *A Kan cubical set X is a cubical set X, equipped with* filling operations $X{\uparrow}$, *and $X{\downarrow}$, such that for every $J, x \in I$, $X{\uparrow}\vec{u}, X{\downarrow}\vec{u}$ are in $X(I)$, where $\vec{u}$ is an open box with u_{x0} and u_{x1} in $X(I - x)$ in the respective cases $X{\uparrow}\vec{u}$ and $X{\downarrow}\vec{u}$, such that the following condition holds:*

$$(X{\uparrow}\vec{u})(y = b) = u_{yb} \text{ and } (X{\downarrow}\vec{u})(x = b) = u_{yb}$$

along with the 'uniformity/coherence' condition, that for $f : I \to K$ defined on J, x, we require:

$$(X{\uparrow}\vec{u})f = X{\uparrow}(\vec{u}f) \text{ and } (X{\downarrow}\vec{u}) = X{\downarrow}(\vec{u}f)$$

We also denote $X^{+}\vec{u} = X{\uparrow}\vec{u}(x = 1)$ and $X^{-}\vec{u} = X{\downarrow}\vec{u}(x = 0)$. This condition for X, known as the *Kan condition* is the analogous statement to *"every horn has a filler"* for Kan complexes, i.e., *"every open box has a filler"*.



Any presheaf category gives rise to a model of dependent type theory by intepreting each presheaf as a type. [7] Hence, cubical sets form such a model with (dependent) products, (dependent) sums, inductive types and identity types. We refine this model by restricting to Kan Cubical Sets taken along with a specified 'Kan Structure', which are the Kan filling operations $X{\uparrow}$ and $X{\downarrow}$. This is needed in order to justify the elimination rules for identity types. [3] Consequently, we can prove functional extentionality, univalence, canonicity for HITs. For a complete, formal treatment, see Simon Huber's PhD Thesis for more details. [9]

References

[1] Thorsten Altenkirch and Ambrus Kaposi. Type theory in type theory using quotient inductive types. *ACM SIGPLAN Notices*, 51(1):18–29, 2016.

[2] Steven Awodey and Andrej Bauer. Propositions as [types]. *Journal of logic and computation*, 14(4):447–471, 2004.

[3] Marc Bezem, Thierry Coquand, and Simon Huber. A model of type theory in cubical sets. In *19th International conference on types for proofs and programs (TYPES 2013)*, pages 107–128. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2014.

[4] Evan Cavallo. *Higher Inductive Types and Internal Parametricity for Cubical Type Theory.* PhD thesis, Carnegie Mellon University, USA, 2021.

[5] Cyril Cohen, Thierry Coquand, Simon Huber, and Anders Mörtberg. Cubical type theory: a constructive interpretation of the univalence axiom. *arXiv preprint arXiv:1611.02108*, 2016.

[6] Alexander Grothendieck. Pursuing stacks. *arXiv preprint arXiv:2111.01000*, 2021.

[7] Martin Hofmann. Syntax and semantics of dependent types. In *Extensional Constructs in Intensional Type Theory*, pages 13–54. Springer, 1997.

[8] William A Howard. The formulae-as-types notion of construction. 1995.

[9] Simon Huber. *A model of type theory in cubical sets.* PhD thesis, Chalmers University of Technology, 2015.

[10] Jacob Lurie. *Higher topos theory.* Princeton University Press, 2009.

[11] Per Martin-Löf. *An intuitionistic theory of types: Predicative part*, volume 80. Elsevier, 1975.

[12] Per Martin-Löf. Constructive mathematics and computer programming. In *Studies in Logic and the Foundations of Mathematics*, volume 104, pages 153–175. Elsevier, 1982.

[13] The Univalent Foundations Program. *Homotopy type theory: Univalent foundations of mathematics.* The Univalent Foundations Program, 2013.

[14] Daniel G Quillen. *Homotopical algebra.* Springer, 2006.

[15] Kristina Sojakova. Higher inductive types as homotopy-initial algebras. In *Proceedings of the 42Nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 31–42, 2015.

[16] Tammo tom Dieck. *Algebraic topology*, volume 8. European Mathematical Society, 2008.

[17] Vladimir Voevodsky. The equivalence axiom and univalent models of type theory.(talk at cmu on february 4, 2010). *arXiv preprint arXiv:1402.5556*, 2014.